随行条件旅游攻略 Agent 系统设计说明书

第 1 章 引言

1.1 编写目的

本文档旨在明确“随行条件”旅游攻略 Agent 的总体架构、模块划分、接口规范与数据结构。作为项目开发、测试与最终验收的核心依据，本文档面向开发团队与课程评审，详细阐述了系统如何通过分层架构与 Agent 技术解决多代同游场景下的行程决策痛点。

1.2 项目背景

传统旅游攻略平台多以单一地点和时间线罗列为主，用户面临“老人腿脚不便、小孩需要母婴室、雨天需要备选方案”等复合条件时，往往需要人工翻阅大量长文本。本系统基于 DeepSeek Agent，通过“结构化预处理 + 条件化检索 + 出处溯源”的技术链路，为用户生成可执行、可核对的个性化行程。

第 2 章 总体设计

2.1 系统架构设计

本系统采用经典的**四层架构**，自顶向下分别为展示层、接口层、业务层与数据/模型层。通过分层设计，实现了前端 UI 与 AI 调用的解耦，提升了系统的可维护性与可扩展性。

1. **展示层（Presentation Layer）**：负责用户交互，包括条件表单输入、基于 SSE 的流式打字机渲染，以及出处浮层展示。
2. **接口层（Interface Layer）**：由 Express 路由构成，负责接收前端 HTTP 请求、参数校验、SSE 事件流推送及标准化错误处理。
3. **业务层（Business Layer）**：系统核心引擎。包含预处理模块（Preprocess）、检索模块（Retrieve）、提示词组装模块（Prompt）及降级处理模块（Fallback）。
4. **数据与模型层（Data & Model Layer）**：包含底层 JSON 结构化片段库以及通过 HTTPS 调用的 DeepSeek API。

2.2 运行环境

- **前端**：现代浏览器（支持 SSE 的 Chrome / Edge / Firefox）。
- **后端**：Node.js (v18+) + Express 框架。
- **大模型**：DeepSeek API（OpenAI 兼容接口格式）。

第 3 章 接口设计

本系统前端与后端、后端与大模型之间通过严格的接口进行通信。

3.1 前后端交互接口（基于 SSE）

为了实现 AI 响应的“打字机”效果，系统采用 SSE (Server-Sent Events) 技术实现流式传输。

- **接口路径**：POST /api/query/stream
- **请求体**：包含城市、天数、同行人员、天气、步行上限等结构化条件。
- **响应事件流**：
    - meta：返回检索到的片段数量与模型名称。
    - chunk：增量文本片段，前端接收后实时追加显示。
    - citation：引用标签信息，前端解析后渲染为可点击的 [片段 N]。
    - done：生成结束标记。
    - error：异常标记，包含降级原因（如 no_result）。

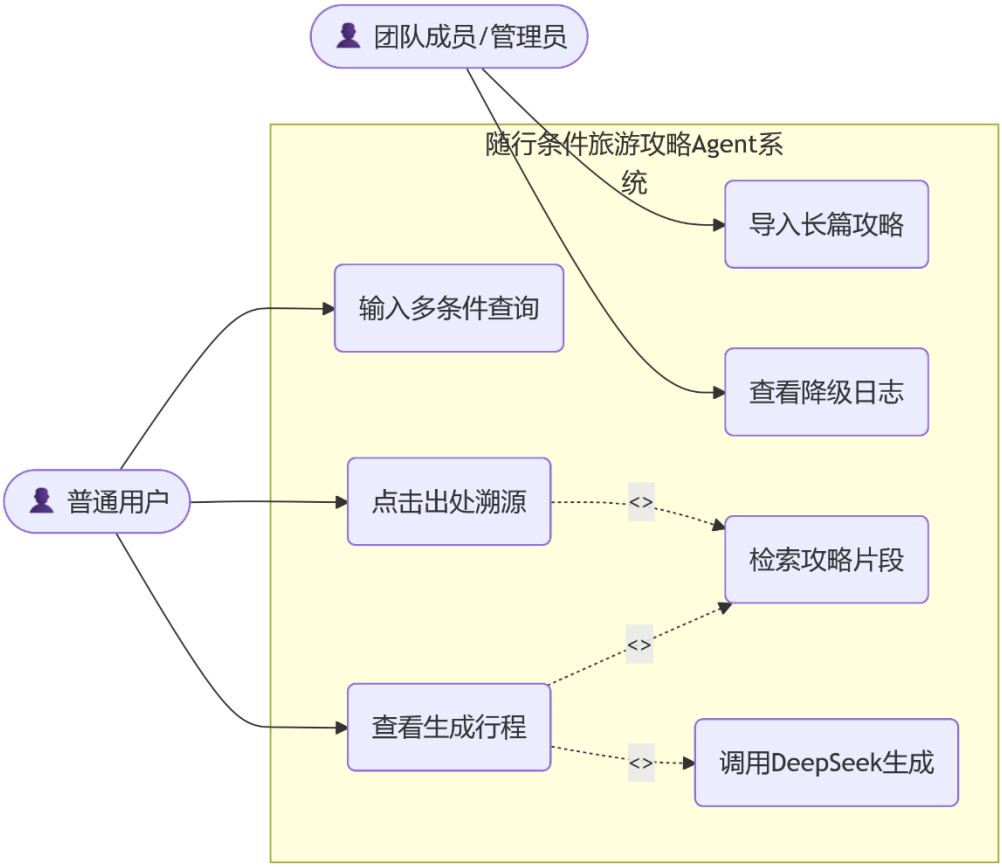
3.2 外部模型接口

- **接口地址**：https://api.deepseek.com/chat/completions
- **鉴权方式**：Authorization: Bearer <DEEPSEEK_API_KEY>（仅后端持有）。
- **超时与降级**：首字返回预期 ≤ 5秒，整体生成超时 60 秒。若超时或网络错误，触发重试或降级返回检索片段摘要。

第 4 章 模块与 UML 设计

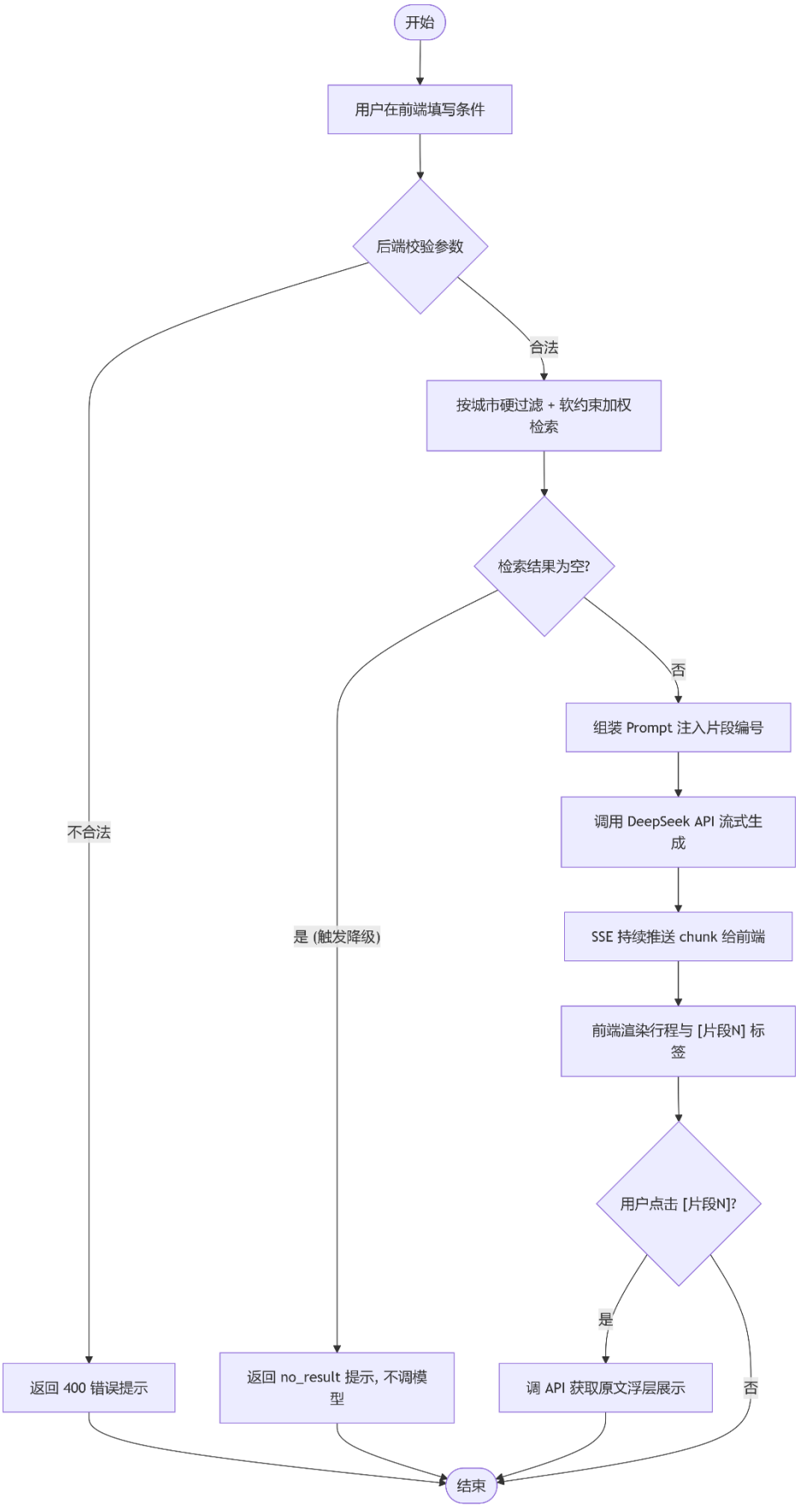
4.1 用例分析

系统主要面向两类角色：普通用户与团队成员（管理员）。普通用户可进行条件查询与出处溯源；管理员负责数据导入与日志监控。



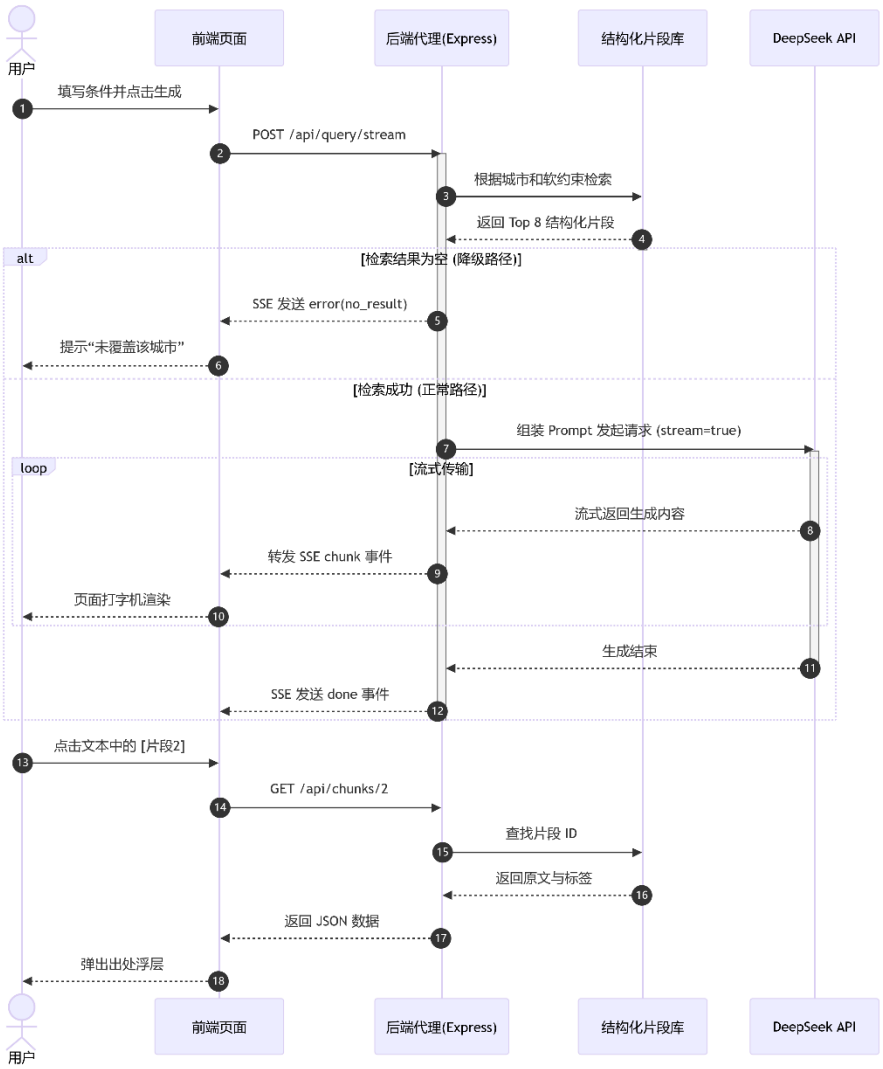
4.2 核心业务流程

用户提交请求后，系统首先进行条件解析与片段检索。若检索为空，直接触发降级提示，不调用大模型以避免“幻觉”编造；若检索成功，则注入 Prompt 调用 DeepSeek 并流式返回。


开始
用户在前端填写条件
后端校验参数
不合法
合法
按城市硬过滤 + 软约束加权检索
检索结果为空?
是 (触发降级)
否
组装 Prompt 注入片段编号
调用 DeepSeek API 流式生成
SSE 持续推送 chunk 给前端
前端渲染行程与 [片段N] 标签
用户点击 [片段N]?
是
否
返回 400 错误提示
返回 no_result 提示, 不调模型
调 API 获取原文浮层展示
结束

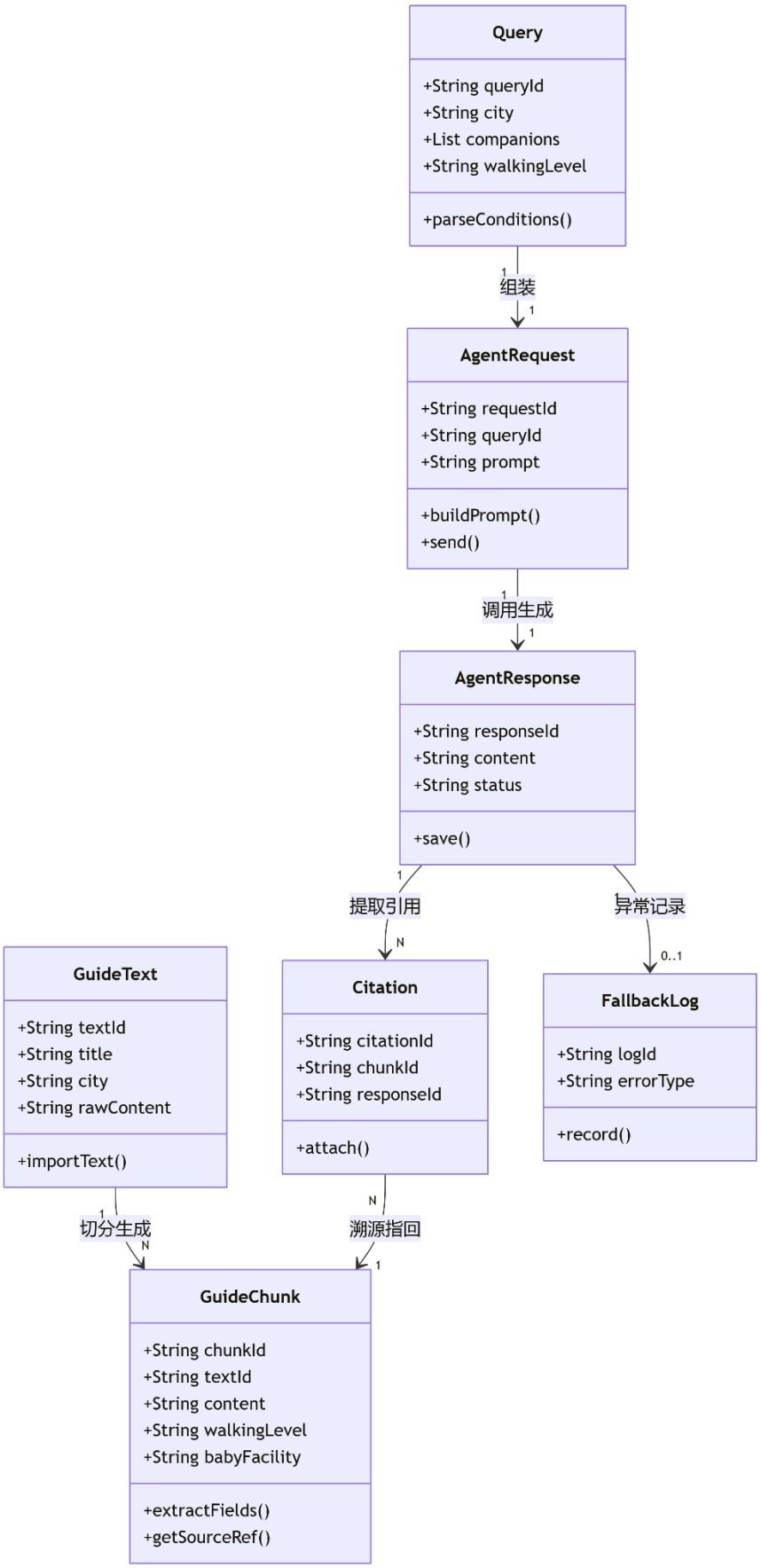
4.3 模块时序交互

从时序图可以看出，Agent 的生成过程并非黑盒。后端在接收到大模型的流式返回后，会实时将其透传为 SSE 事件。用户点击引用时，前端通过独立接口二次请求底层原文，实现了“生成”与“溯源”的分离。



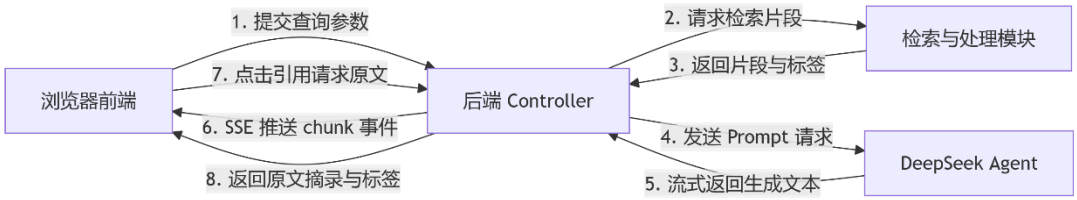
4.4 核心类图设计

系统的对象模型围绕“可追责”设计。AgentResponse（响应）通过 Citation（引用）类，精准映射回底层的 GuideChunk（结构化片段）与 GuideText（原文），确保任何一段生成文本都能找到数据依据。


Query
+String queryId
+String city
+List companions
+String walkingLevel
+parseConditions()
1
组装
1
AgentRequest
+String requestId
+String queryId
+String prompt
+buildPrompt()
+send()
1
调用生成
1
AgentResponse
+String responseId
+String content
+String status
+save()
1
提取引用
N
异常记录
0..1
GuideText
+String textId
+String title
+String city
+String rawContent
+importText()
Citation
+String citationId
+String chunkId
+String responseId
+attach()
FallbackLog
+String logId
+String errorType
+record()
1
切分生成
N
N
溯源指回
1
GuideChunk
+String chunkId
+String textId
+String content
+String walkingLevel
+String babyFacility
+extractFields()
+getSourceRef()

4.5 协作关系

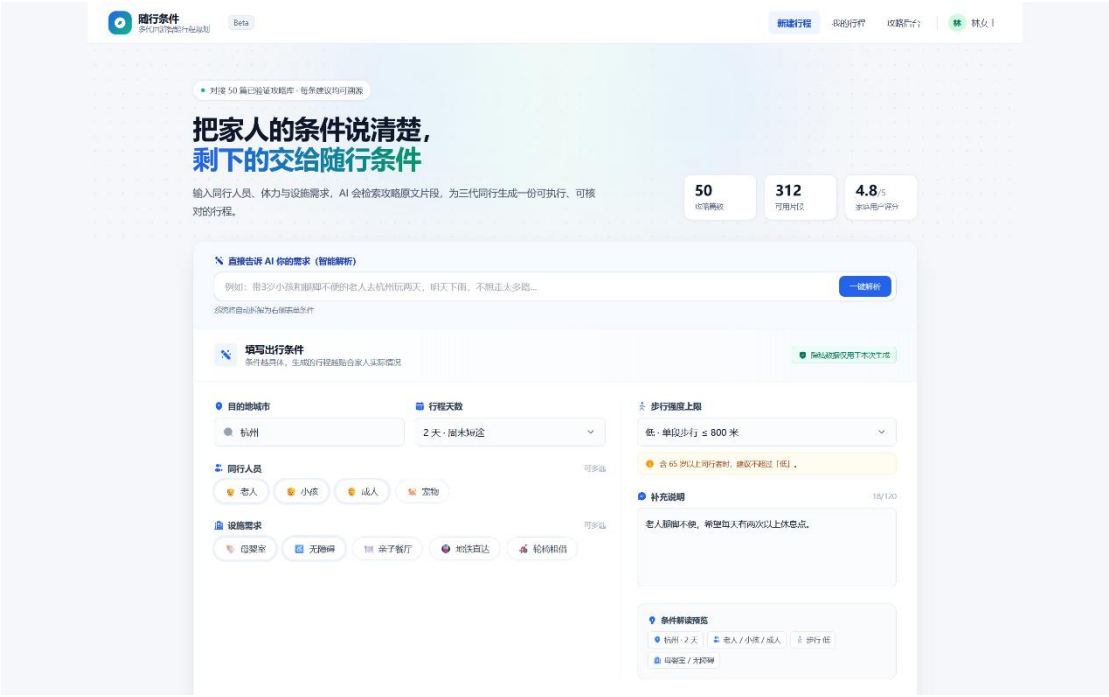
各模块在底层通过协同工作完成一次完整的生成任务。前后端、检索模块与 Agent 服务严格按照编号 1-8 的消息序列完成数据流转。



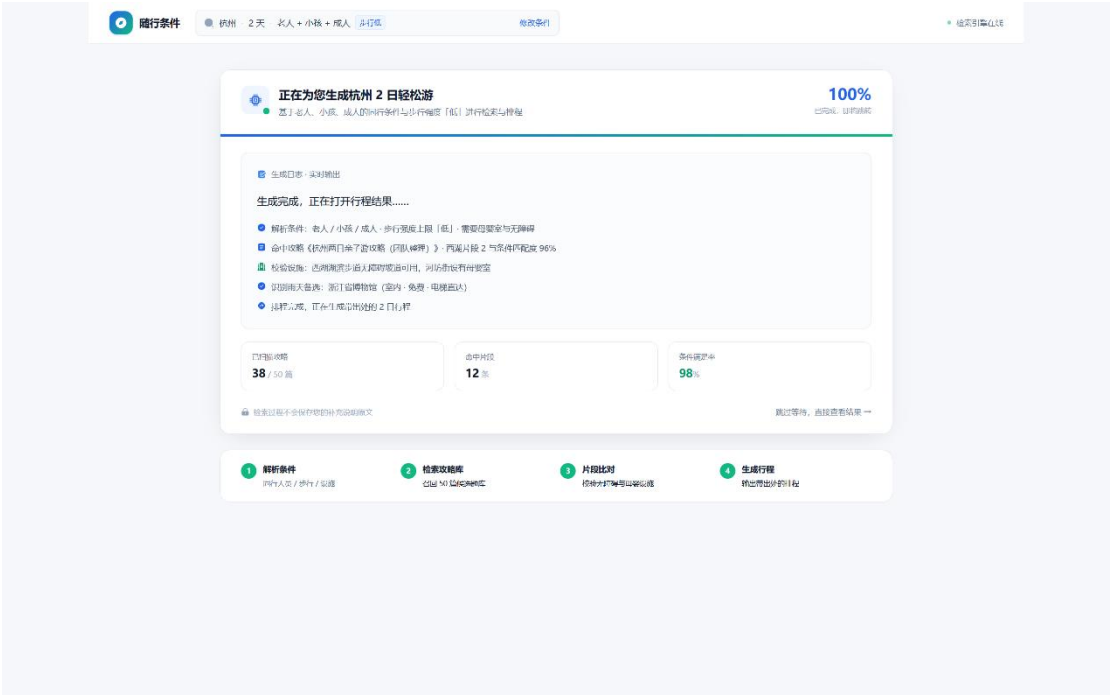
第 5 章 界面与原型设计

本系统前端采用高保真交互原型设计，核心链路闭环如下：

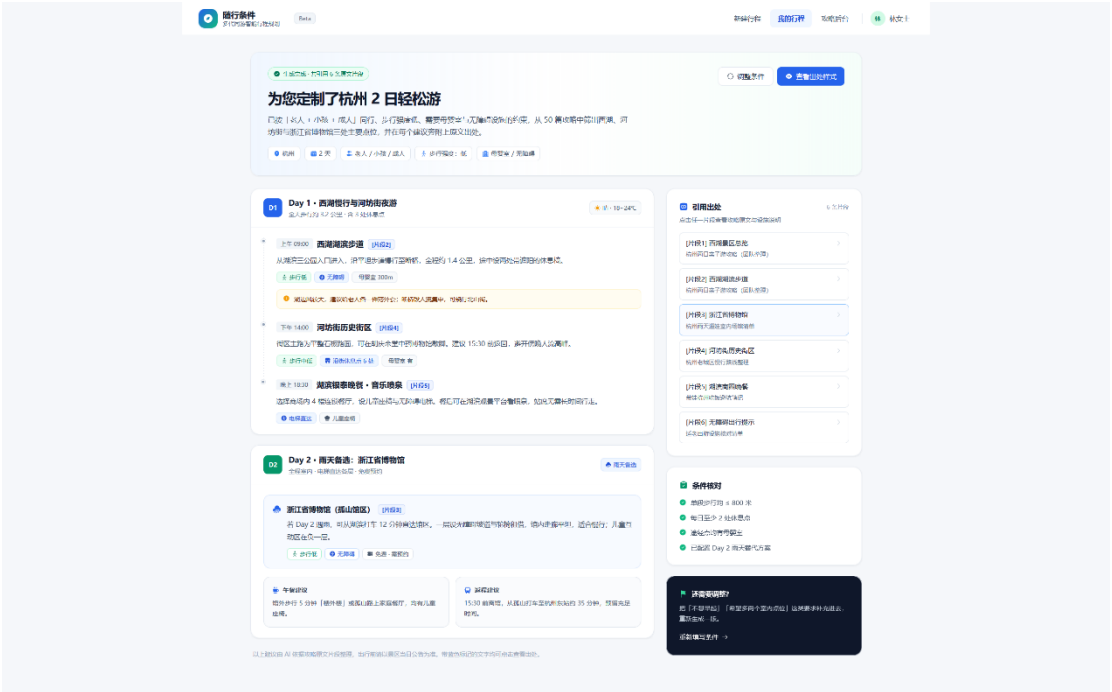
1. **首页（条件输入）**：支持硬/软约束的复合选择（如多代同行、设施需求），提供明确的表单交互。



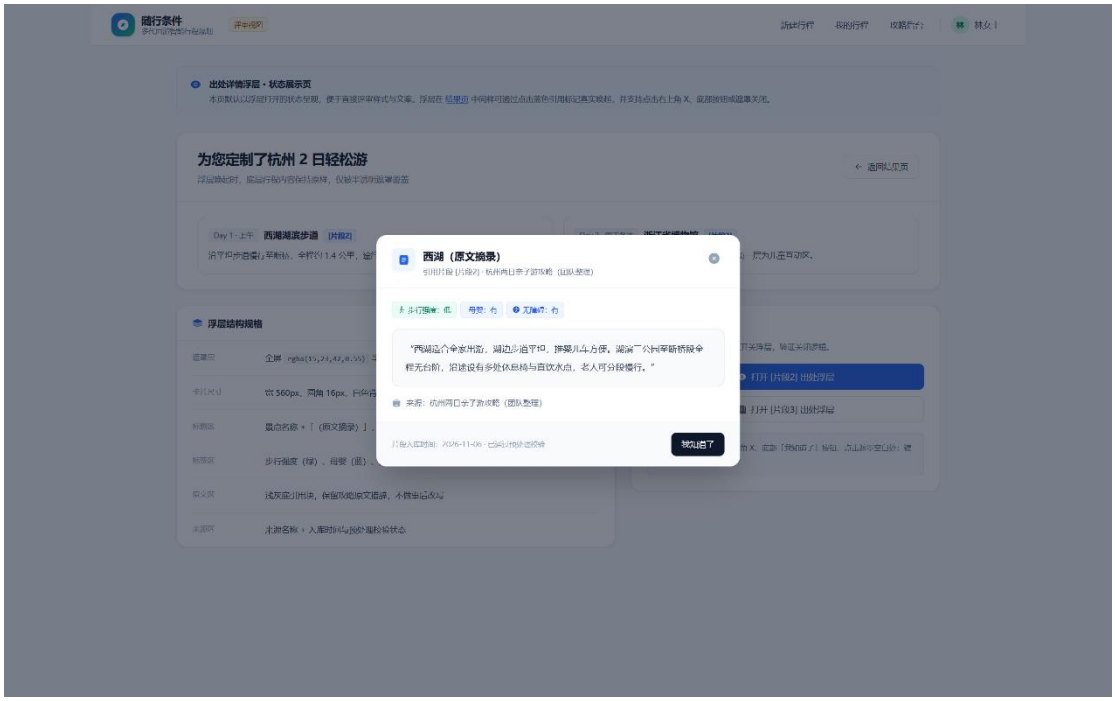
2. **生成等待页**：模拟并展示 Agent 的工作链路。通过动态进度条与检索日志（命中攻略、设施校验），向用户透明化地展示 AI 的思考与召回过程。



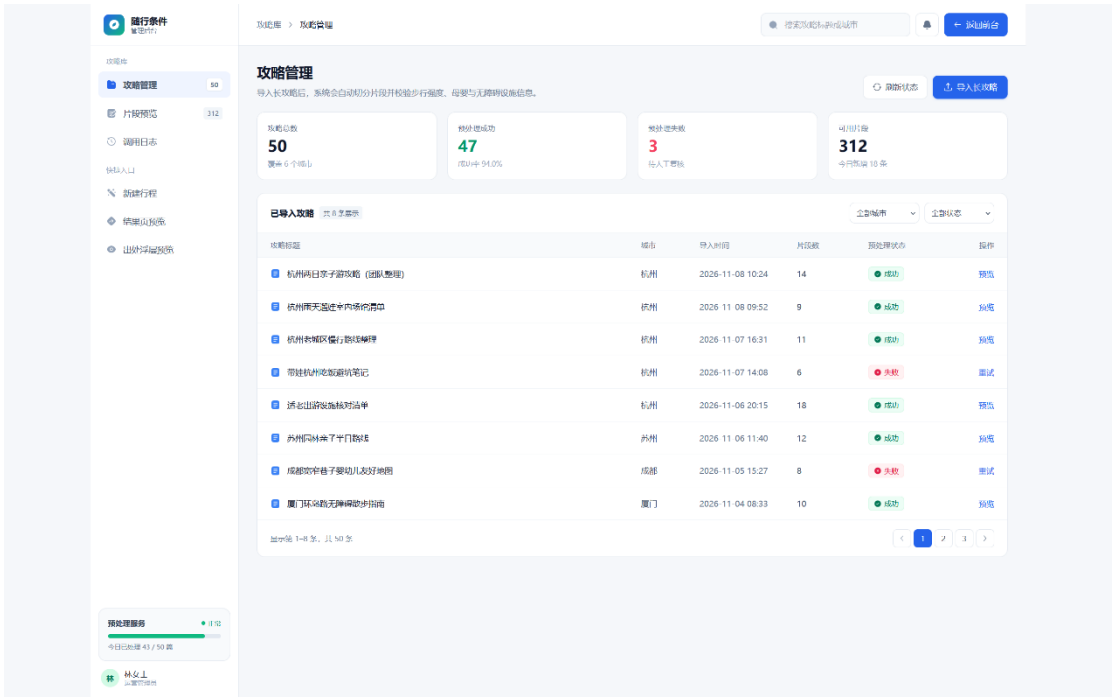
3. **结果展示页**：输出结构化的时间轴行程。左侧为建议正文，右侧为引用面板。文本中嵌入蓝色 [片段 N] 可点击标签。



4. **出处详情浮层**：点击标签后唤起。展示攻略原文摘录、来源说明以及步行强度、母婴、无障碍等关键结构化标签，彻底解决 AI 幻觉问题。



5. **后台管理页**：供团队内部使用。监控攻略预处理状态（成功/失败），并查看 Agent 调用的降级日志与耗时分析。



第 6 章 异常处理与系统降级设计

系统设计了三种核心异常路径，确保生成过程的安全可控：

1. **检索为空（未覆盖城市）**：系统拦截查询请求，**不调用大模型生成**，直接向用户返回“当前攻略库未覆盖该城市”的提示，杜绝模型无中生有。

2. **大模型超时/限流**：触发重试机制。若重试失败，写入 FallbackLog，前端展示降级提示，并展示已检索到的攻略片段摘要。
3. **数据溯源失败**：若引用的片段 ID 映射失效，浮层展示“出处暂不可用”，并提示用户该条建议仅供参考。